ChatGPT

Executive Summary

Shizuku is a system that lets apps run privileged code (as the shell/ADB user) via a special Binder interface【29†L54-L64】. In practice, an app installs the Shizuku app, starts it (via USB or wireless debugging or root), and then uses the Shizuku library to bind to protected Android services. When done correctly, this allows call-recording apps to access audio capture services with near-system permissions. The ACR Phone Helper (APH) app has successfully used Shizuku to record *both sides* of a call (including Bluetooth/headphone calls) on Android 10+【3†L30-L39】【44†L310-L319】. By contrast, your helper app is failing to record both sides, which suggests it has not properly obtained or used the Shizuku Binder/privileges.

The solution is to follow the standard Shizuku integration pattern: add the required permissions and provider to your manifest, start Shizuku via ADB or wireless (especially on Android <11), request the user's permission within your app, and then wrap the target system service Binder with `ShizukuBinderWrapper`【29†L133-L142】. In code, one typically does something like:

```
if (!Shizuku.pingBinder()) {
    Shizuku.addBinderReceivedListener(() -> initRecording());
} else {
    initRecording();
}
// ...
void initRecording() {
    if (Shizuku.checkSelfPermission() != PackageManager.PERMISSION_GRANTED) {
        Shizuku.requestPermission(REQUEST_CODE);
        return;
    }
    // Now we have permission: get the audio service via Shizuku
    IBinder audioBinder =
SystemServiceHelper.getSystemService(Context.AUDIO_SERVICE);
    IAudioService audioSvc = IAudioService.Stub.asInterface(
        new ShizukuBinderWrapper(audioBinder));
    // Use audioSvc (hidden API) or start a privileged recording.
}
```

Key checks: verify Shizuku is running (`adb shell service list | grep shizuku`), that the Shizuku permission was granted, and that your app binds correctly. Using logcat (e.g. filtering for `Shizuku` and `AudioRecord`) and `dumpsys` is critical to debug binding. The differences between APH and your app likely lie in these steps: APH's own "Shizuku fork" may include fixes, but more importantly APH surely performs the Shizuku permission request and binder-wrapping correctly【3†L59-L67】【29†L133-L142】.

This report covers Shizuku architecture and APIs, common binding patterns, step-by-step integration guidance (with manifest entries and code examples), troubleshooting tips (ADB/logcat commands, common errors), device/OEM quirks, and a comparison of recording methods (accessibility vs Shizuku

vs root/OEM). We also summarize evidence from open-source projects (e.g. ShizuCallRecorder) on how real two-way recording is achieved via Shizuku【3†L30-L39】【44†L310-L319】.

1. Shizuku Architecture and APIs

Overview: Shizuku runs a *privileged* Java process (UID 2000, the "shell" user) that exposes many Android system APIs over Binder【29†L54-L64】. An app can then proxy Binder calls to those system services as if it were running with ADB/shell privileges (but without full root). Because the Shizuku process runs with the ADB/shell identity, it inherits Android system permissions typically granted to shell (e.g. many system APIs)【29†L58-L64】. However, it is still subject to normal Android SELinux and UID checks, so not *everything* is possible (e.g. APIs restricted to signature-level permission or requiring the system UID will still fail).

Startup modes: There are three ways to start Shizuku: (a) via ADB shell (wired USB or network) using a one-line script (`adb shell sh /sdcard/Android/data/moe.shizuku.privileged.api/start.sh`)【29†L89-L98】, (b) via Wireless debugging (Android 11+) from the Shizuku UI【29†L82-L90】, or (c) on a rooted device via `su` and the Shizuku start script【29†L96-L100】. On Android 6–10, only USB ADB is available (manual restart on each boot)【1†L315-L318】; on 11+ wireless debug can start Shizuku without a PC【1†L315-L318】. You must ensure Shizuku is actually running for your recording to work. After reboot, users typically need to re-open Shizuku (or re-run the ADB command) to relaunch the privileged service【3†L71-L74】.

Shizuku Versions: The official Shizuku app on the Play Store lags behind latest versions【3†L53-L58】. In fact, NLL Apps (developers of APH) maintain their own fork of Shizuku optimized for APH【3†L85-L94】. Their fork builds on the popular "thedjchi/Shizuku" which may include audio/capture fixes. When developing, you can use either the official Shizuku or thedjchi/NLL forks; ensure your code handles any version-specific differences (most modern apps target the Shizuku v11+ API). According to the Shizuku API docs, your app should add Gradle dependencies for Shizuku API (and provider) to use Shizuku【1†L340-L349】.

Permissions & Manifest: To use Shizuku, your app's manifest must declare:

```
<uses-permission android:name="moe.shizuku.manager.permission.API"/>

<provider
    android:name="rikka.shizuku.ShizukuProvider"
    android:authorities="${applicationId}.shizuku"
    android:enabled="true"
    android:exported="true"
    android:permission="android.permission.INTERACT_ACROSS_USERS_FULL"
    android:multiprocess="false"/>
```

This matches examples in the Shizuku guide【29†L133-L142】. The special `INTERACT_ACROSS_USERS_FULL` permission on the provider allows inter-process communication.

API Flow: In code, you typically do the following【29†L147-L156】【29†L153-L142】: 1. **Initialize and wait:** Call `Shizuku.addBinderReceivedListener(...)` or poll `Shizuku.pingBinder()` to know when the Shizuku binder is available. (On some versions you use `ShizukuServiceConnection`; the

library guides cover this.) 2. **Check Shizuku permission:** Once binder is alive, call `Shizuku.checkSelfPermission()`. If not granted, call `Shizuku.requestPermission(reqCode)`. The user must then approve in the Shizuku UI, and you handle the result in `onRequestPermissionsResult()` as needed. 3. **Wrap system service binder:** After permission is granted, fetch the system service's binder and wrap it. For example, to access the package manager:

```
IBinder binder = SystemServiceHelper.getSystemService("package");
IPackageManager pm = IPackageManager.Stub.asInterface(new
ShizukuBinderWrapper(binder));
```

(This example is from HackTricks【29†L147-L156】.) For call recording you would fetch the audio or telephony service. In Android, audio services like `Context.AUDIO_SERVICE` or `MediaRecorderService` can be accessed via hidden APIs through their AIDL interfaces. By wrapping their Binder with `ShizukuBinderWrapper`, your calls to these services go through the Shizuku process as shell【29†L147-L156】.

Key Points:

- Shizuku is not the same as root: it simply runs as the shell user (UID 2000). Some privileged APIs still require signature-level or system-level certs.
- The binder you get via Shizuku is exactly the remote `IBinder` of the target system service, so you use normal AIDL stubs (e.g. `IAudioService.Stub.asInterface(...)`) on it.
- Always handle `Shizuku.addBinderDeadListener` in case the service dies (e.g. if Shizuku process stops).
- Remember to call `ShizukuServiceConnection.shutdown()` or similar if using that approach, to cleanly unbind.

2. Binding to Privileged Services

AIDL and Binder: Android system services often define AIDL interfaces (like `IPackageManager`, `IAudioService`, etc.). Normally, apps without system permission cannot bind to most of these. Shizuku's trick is that it uses a hidden ContentProvider (`ShizukuProvider`) to bootstrap a connection to the `moe.shizuku.privileged.api` service. Once you have the Shizuku `IBinder`, you can ask for *any* other system service's binder by name (via `SystemServiceHelper`) and execute transactions. Under the hood, Shizuku inserts itself into the binder IPC path, giving your app a proxy to the service as if you had shell rights.

ServiceConnection: Note that this is not the usual `bindService()` approach. You do not create an Intent or subclass of Service. Instead, you directly call `SystemServiceHelper.getSystemService(name)` to retrieve the `IBinder`. This is a synchronous call via Shizuku's ContentProvider. The AIDL stub is then created on the client side. For example:

```
Shizuku.addBinderReceivedListener(() -> {
    if (Shizuku.checkSelfPermission() == PackageManager.PERMISSION_GRANTED) {
        // get audio service binder
        IBinder audioBinder =
```

```
SystemServiceHelper.getSystemService(Context.AUDIO_SERVICE);
        IAudioService audioService = IAudioService.Stub.asInterface(new
ShizukuBinderWrapper(audioBinder));
        // Now you can call audioService.startVoiceRecording(...) or similar
hidden methods
    }
});
```

Some apps may still use `Context.bindService()` if a system service offers a public binding (e.g. with action and permission), but for call recording typically we use the direct AIDL route.

pingBinder: It's often helpful to call `Shizuku.pingBinder()` to check if the Shizuku binder is already available; if it returns false, call `addBinderReceivedListener` to wait. This prevents IllegalStateException.

SELinux/UID constraints: Even with Shizuku, calls are still subject to Android's permission model. For example, some audio-recording APIs require `android.permission.RECORD_AUDIO` or special system permissions. However, since you run as shell, you can often bypass user-facing restrictions (e.g. background limits, security flags). **Cally** (an advanced Shizuku-based recorder) notes that they must verify calling UID matches the app and signature hash on each AIDL call to prevent abuse【24†L638-L647】. In most helper apps you just rely on Shizuku's security prompt and do not implement extra checks.

3. Integration Steps for an Android Helper App

To connect Shizuku to your helper app and start two-way call recording, follow these steps:

1. **Install Shizuku app:** Instruct the user to install Shizuku (official or a recommended fork) from Play Store or other source. Have them open Shizuku and start it via ADB or wireless debugging. (Provide guidance or link to Shizuku's guide【27†L53-L61】.)

2. **Enable Developer Options:** On Android 10 and below, enable USB debugging and use `adb` to run the Shizuku start script【27†L172-L180】. On Android 11+, enable Wireless Debugging.

3. **Add Manifest entries:** In your helper app's `AndroidManifest.xml` include:

```
<uses-permission android:name="moe.shizuku.manager.permission.API" />
<provider
    android:name="rikka.shizuku.ShizukuProvider"
    android:authorities="${applicationId}.shizuku"
    android:enabled="true"
    android:exported="true"
    android:permission="android.permission.INTERACT_ACROSS_USERS_FULL"
    android:multiprocess="false" />
```

Also ensure you request any other needed permissions (e.g. RECORD_AUDIO, MODIFY_AUDIO_SETTINGS) in the manifest.

1. **Gradle dependency:** Include Shizuku API in your build (for example, `implementation "dev.rikka.shizuku:api:<version>"` and `...:provider:<version>"` as per [1†L340-L349]).

2. **Initialize Shizuku in code:** In your main Activity or service, add code to acquire the Shizuku binder:

```
// e.g. in onCreate or onResume
if (!Shizuku.pingBinder()) {
    Shizuku.addBinderReceivedListener(new Shizuku.BinderReceivedListener() {
        @Override
        public void onBinderReceived() {
            initShizuku();
        }
    });
} else {
    initShizuku();
}
```

In `initShizuku()`, check permission and call privileged APIs:

```
void initShizuku() {
    if (Shizuku.checkSelfPermission() != PackageManager.PERMISSION_GRANTED) {
        Shizuku.requestPermission(REQUEST_SHIZUKU);
        return;
    }
    // Shizuku permission granted
    startPrivilegedRecording();
}
```

1. **Request permission:** When `requestPermission()` is called, the user will see a Shizuku dialog. Override `onRequestPermissionsResult(...)` to detect when granted, then proceed. You can also use `Shizuku.addBinderReceivedListenerSticky` to catch the event after grant.

2. **Bind to the recording service:** Once permission is granted, call the hidden recording APIs. There are two main strategies:

3. **Using IAudioService / MediaRecorderService:** For true dual-stream recording, one might obtain the `IAudioService` binder and use methods like `startVoiceCommunication` or start a raw AudioRecord. This typically requires hidden APIs. Example:

```
IBinder audioBinder =
SystemServiceHelper.getSystemService(Context.AUDIO_SERVICE);
```

```
IAudioService audioService = IAudioService.Stub.asInterface(new
ShizukuBinderWrapper(audioBinder));
// e.g. call audioService.startVoiceCommicationStream() or similar
```

(The exact method depends on Android version and OEM.)

4. **Using a bound "recorder" service:** Alternatively, mimic apps like **Cally** by starting a `UserService` process under Shizuku that implements an `IRecorderService` AIDL. That service then does the actual AudioRecord capturing with `VOICE_CALL` source. In either case, the core idea is the same: the recording code runs with shell privileges via Shizuku.

5. **Error handling and logging:** Include try/catch around all Shizuku calls. For example:

```
try {
    // binder calls
} catch (RemoteException | SecurityException e) {
    Log.e(TAG, "Shizuku error: " + e);
}
```

Also log progress: e.g. "Shizuku binder received", "Shizuku permission granted", "Starting recording service", etc. Use `Log.i` for key events, so you can filter via logcat (`logcat | grep Recording` or `logcat | grep Shizuku`).

1. **Handle Shizuku restarts:** If the device reboots or Shizuku stops, you may need to rebind. Use `Shizuku.addBinderDeadListener(...)` to know if Shizuku's process died and attempt to re-init. In practice, just show the user a notification to restart Shizuku and re-enable recording.

2. **Testing code:** To verify the binder connection, you can call `Shizuku.pingBinder()` and log the result. For example:

```
boolean alive = Shizuku.pingBinder();
Log.i(TAG, "Shizuku binder alive? " + alive);
```

After requesting permission, check `Shizuku.checkSelfPermission()` again. You should see "PERMISSION_GRANTED" in log after the user approves Shizuku.

4. Troubleshooting Checklist

If two-way recording still fails after integration, debug with these steps:

- **Verify Shizuku is running:** On your computer:

```
adb shell service list | grep shizuku
```

You should see `moe.shizuku.privileged.api` listed. If not, start Shizuku (wireless or ADB).

- **Check Shizuku permission:** On device, Shizuku app shows which apps have permission. Ensure your helper is listed under "Granted Apps." In code, after `requestPermission()`, print `Shizuku.checkSelfPermission()`.

- **Check provider authorites:** Ensure the manifest authority matches exactly `<your app ID>.shizuku`. Mis-naming it can prevent the ShizukuProvider from initializing.

- **Logcat filters:** Watch logcat for errors. For example:

```
adb logcat | grep Shizuku
adb logcat | grep Recording
adb logcat | grep AudioRecord
```

Shizuku library may log binding actions, and `AudioRecord` logs can show if recording started.

- **dumpsys and debug:** Use dumpsys to diagnose:

```
adb shell dumpsys activity services moe.shizuku.privileged.api
adb shell dumpsys media.audio_flinger
```

This can show if a recording thread is active or if the audio flinger has no client.

- **Battery/Background:** On some OEMs (Xiaomi, Samsung, etc.), background apps or services get killed aggressively. Ensure battery optimizations are disabled for both Shizuku and your helper【3†L75-L78】. Lock them in "protected apps" or whitelist as needed.

- **ADB Logs on Crash:** If Shizuku or your app crashes, view stack traces. The Reddit thread notes Shizuku might crash on calls (e.g. due to mic conflicts)【21†L184-L193】. Filtering logcat by `Shizuku` can show if it's stopping unexpectedly.

- **Permission Errors:** If you see "permission denied" in logs, check that you also have RECORD_AUDIO and MODIFY_AUDIO_SETTINGS in your manifest, since capturing call audio typically requires these.

- **Call State:** Some recorders only start when a call is *active*. Ensure your code triggers recording on `TelephonyManager` or similar. You may need a PhoneStateListener or BroadcastReceiver for `ACTION_NEW_OUTGOING_CALL` to know when to start the capture.

- **Check for Other Apps:** If another recording app or service is running, it might block the microphone. Close any other call recorder or VoIP app.

Common failure modes & fixes:

- *"Shizuku not found/connected"*: Shizuku app not running or no permission. Remedy: Start Shizuku (via `adb` or wireless) and re-grant permission.

- *"No audio from one side"*: Often a device codec issue (see below). Verify by trying different methods (wired vs. BT vs. speakerphone).
- *"Shizuku permission dialog not appearing"*: Ensure you called `requestPermission()` on the UI thread and have the correct provider. You may need to restart the app after manifest changes.

5. Device/OEM Quirks and Workarounds

Call recording support varies widely by device/Android version. Known issues include:

- **Hardware Codecs (Realtek/MediaTek):** Some chipsets limit capturing uplink audio. For example, older Samsung and Realtek-based phones could not record the other side at all. APH's statement "two-way (including Bluetooth/headset)" suggests Shizuku can bypass some of these, but it may still fail on certain OEMs. If you see only one side, it's possibly a hardware restriction. Workarounds include using speakerphone or relying on the newer Android 10 audio capture APIs (see next point).

- **Android 10+ Restrictions:** Google locked down call audio. In Android 10/11, only system apps with special privileges (e.g. AOSP dialer) can record calls. However, Shizuku "impersonation" can simulate a system context. The open-source *cally* and *ShizuCallRecorder* use tricks like `AudioRecord.Builder.setAudioSource()` with `VOICE_CALL` or using a fake `Context` to bypass hidden API checks【24†L708-L715】. Our helper app may need to use similar hidden calls. For example, on Android 10+ use `AudioRecord.setAudioSource(MediaRecorder.AudioSource.VOICE_COMMUNICATION)` or `AudioSource.VOICE_CALL` via hidden API to capture both streams.

- **Bluetooth Calls:** Typically, when on Bluetooth, the phone audio route changes and VOICE_CALL stream goes to the headset. Some apps cannot capture Bluetooth audio. However, APH and ShizuCall suggest Shizuku can capture BT calls【3†L30-L39】. Test on and off BT. If BT calls fail, it may be an OS-level block; the only known workaround is to use wired connection or speakerphone.

- **Speaker vs Handset:** Some devices record better on speakerphone. If downlink is absent, advise user to switch to speaker mode (temporarily) to see if it helps.

- **Wi-Fi Calling / VoLTE:** These often use a different audio path. Many 3rd-party recorders cannot capture Wi-Fi calls at all【3†L79-L81】. The APH notes this is common: "It may not be possible to record both sides on some phones when Wi-Fi calling is used... This is usually due to OEM audio drivers【3†L79-L81】." The workaround is to disable Wi-Fi calling during recording.

- **OEM "Native" Recording:** Some manufacturers (Samsung, Xiaomi, OnePlus) have their own call-recording dialers. These are signature-privileged. Shizuku cannot directly enable them, but sometimes you can call hidden vendor APIs. This is complex and often device-specific; skip it unless needed.

- **Enhanced Privacy Mode:** Newer Android versions (14/15) add "Enhanced Confirmation Mode" that may detect audio capture hacks【24†L668-L677】. This could break Shizuku-based recorders on future OS. Keep libraries updated.

- **Battery and Background:** As noted, aggressive task killers can stop Shizuku or the recording service. Use Android's foreground service for recording if possible (show a notification) to reduce risk of being killed.

If encountering any of these quirks, check XDA or device forums for model-specific advice. In summary, two-way recording on Android is inherently fragile. What works on one phone may fail on another; the Shizuku method is currently among the best solutions, but still not guaranteed on all hardware【3†L30-L39】【44†L310-L319】.

6. APH's Shizuku Integration (vs. Typical Helper)

We could not find explicit source code in APH's repository for the Shizuku pathway (the open-source APH code seems to focus on Accessibility mode). However, from NLL Apps' documentation and example projects, we infer APH does the following differently:

- **Proper Shizuku binder usage:** APH likely includes the Shizuku API library and its manifest has the provider and permission. In contrast, your helper may have missed a provider or uses a wrong authority. Double-check that the Shizuku permission line is present in your manifest 【29†L133-L142】.

- **User flow:** APH's instructions say "Open APH and tap the on-screen button to grant Shizuku permission when prompted"【3†L65-L69】. This implies APH explicitly calls `requestPermission()` and waits for approval. Ensure your app is doing the same (some apps forget to handle the permission prompt).

- **Shizuku fork features:** The NLL Shizuku fork may include audio-related enhancements. For example, it might unlock hidden APIs or adjust SELinux. Using APH's recommended Shizuku fork ("thedjchi" or their own) may help. Ensure you test with their fork if the official one fails.

- **Recording implementation:** APH must invoke a privileged audio capture. Even without source, think like the ShizuCallRecorder: APH likely starts an Android Service (FGS) or similar under Shizuku. Check if APH shows a persistent notification when recording (common technique to avoid background death). If your helper doesn't, try adding a foreground service around the capture code.

- **Caller verification:** Unlike our helper, APH may check for active call state before starting Shizuku APIs. Ensure your app only tries to start recording when a call is active, otherwise Shizuku may not provide audio (some APIs only work during a call).

- **Summary:** We found no direct code differences (the APH source repository doesn't mention Shizuku). But the behavior strongly suggests APH correctly binds to the privileged recording service via Shizuku. Compare your code to known examples (e.g. ShizuCallRecorder's use of Shizuku)【29†L147-L156】【44†L310-L319】. The fix is not just conceptual – it's likely a missing line of code or manifest entry.

7. Security, Compatibility and Legal Considerations

- **Security Model:** Using Shizuku elevates your app's capabilities to that of the shell user. *User consent is mandatory* – they must enable Developer Options and explicitly grant Shizuku permission. This reduces risk: a random app cannot silently record calls; the user must set it up. However, warn users: enabling USB debugging or wireless debug can be a security risk if other malicious apps exist. Only recommend Shizuku to trusted power users.

- **App Permissions:** Even with Shizuku, Android's permission model still applies. You must still request `RECORD_AUDIO`, `WRITE_EXTERNAL_STORAGE` (if saving files), etc. Shizuku does not bypass these; it only provides additional system-level permissions. For example, if your code uses AudioRecord, it still needs `RECORD_AUDIO`. Make sure to request and check these at runtime.

- **Android Version Compatibility:** Our solution should target Android 10+ (since APH's Shizuku method is for 10+). Below Android 10, Google's official VOICE_CALL MediaRecorder API was available for system apps (but removed later). On Android 11+, accessibility recording is banned on Play Store【4†L142-L149】, so third-party apps must use Shizuku or root. Future Android releases may break Shizuku hacks (for example, Android 16 may restrict fake package context tricks【24†L662-L670】). We mention these as risks.

- **Legal:** Be aware of call-recording laws. In many jurisdictions, recording requires one or all-party consent. Even if technically possible, it might be illegal without notice. We do *not* condone illegal use; include a warning that users are responsible for complying with local laws. (For example, US/EU often need both parties informed, whereas UK and India allow one-party recording.)

- **Google Play Policy:** Google disallows any non-system app from recording calls via the call log or microphone unless it's the default phone app with user consent. APH itself had to be on F-Droid/NLL Store because Play Store banned accessibility call recorders【4†L142-L149】. If you plan to distribute via Play, know that mentioning "call recording" in Play Store descriptions is currently prohibited. Many devs hide it or publish on alternative stores.

- **Battery/Performance:** Running long AudioRecord or MediaRecorder sessions in background is power-intensive. Make sure to stop recording promptly when the call ends. Use a `ForegroundService` during the call (with a notification) to avoid being killed by the system.

8. Methods Comparison

Method	Requirements	Privileges Granted	Bluetooth/ Headset	Reliability & Quality	Pros	Cons
Accessibility Service	Accessibility access, Android 10+ only (Android APIs) 【4†L142-L149】	None (uses mic only)	No (only mic)	Poor (only mic capture; other side faint or silent)	Works without debugging tools; no root needed	Blocked on Play Store; Google bans it; low quality on new Android
Shizuku (ADB)	Developer mode, Shizuku app, ADB/wireless setup	Shell user (~root) via Binder 【29†L54-L64】	Yes (can capture via privileged APIs) 【3†L30-L39】 【44†L310-L319】	Good (true two-way possible, high quality)	No root/ unlock required; captures both sides including BT	Complex setup (Dev options); may break on some OEMs; needs re-authorising each boot
Magisk/Sui (root)	Rooted device or Magisk	Full root (if use system app)	Yes	Very good (full access)	Strong privileges, auto-start on boot	Requires root (not common for average user); root could affect security
OEM Native API	Specific OEM phone, default dialer	System/ privileged (OEM)	Depends on OEM	Excellent (usually)	Official, usually no user setup	Only available on some phones; may add beep or require region unlock
Workaround (e.g. speakerphone)	None (just hardware trick)	No special Android privileges	Yes (via workaround)	Variable (extra noise, not true solution)	No software change, always works physically	Very hacky; low audio quality; inconvenient for user

Table notes: "Quality" refers to how clearly both sides are captured. Accessibility-only methods typically yield the caller's voice only (uploader or sidetone), whereas Shizuku or root allow true dual-stream capture. The official OEM method (like Samsung's) is best quality but only on supported devices.

9. Stepwise Test Plan

To validate two-way recording with Shizuku, follow this plan:

1. **Setup:** Enable Developer Options on the phone. Install Shizuku (and its recommended fork if needed). Connect via USB debug or wireless. Run `adb shell sh /sdcard/Android/data/moe.shizuku.privileged.api/start.sh` (or use Shizuku UI) and confirm service is running (`adb shell service list | grep shizuku`).

2. **Install Helper App:** Build and install your helper app with the Shizuku integration. Grant all normal permissions (Audio, Storage).

3. **Grant Shizuku:** Open your helper app. It should detect Shizuku running. Tap any "Enable" button to trigger `Shizuku.requestPermission()`. On the phone, accept the Shizuku permission dialog.

4. **Verify Binder:** After granting, in `onResume()` check `Shizuku.checkSelfPermission()` and log it (should be `GRANTED`). Also log `Shizuku.pingBinder()` (should be true). In logcat you should see something like "Shizuku: permission granted" if you log it.

5. **Make a Call:** Use the dialer to call any number (or have someone call you). During the call, your app should start recording (e.g. as soon as call is connected or when a "Start Recording" button is pressed).

6. **Check recording:** After ending the call, stop the recording service. Verify that an audio file was saved. Play it back and confirm you hear *both* voices clearly (even with Bluetooth/headset if those were used).

7. **Logcat Inspection:** While repeating a test call, monitor logcat on your PC:

```
adb logcat *:S ShizukuBinderWrapper:I AudioRecord:I
```

You should see logs indicating ShizukuBinderWrapper usage and AudioRecord started. For example, a message like "AudioRecord started" and "code=2" or so, indicating channels. If no second track is present, check for warnings or permission errors.

1. **Test Variations:** Try calls on speakerphone, Bluetooth headset, and wired earphones. Note differences. Also test an emergency like incoming call when the app is in background (should still record if coded correctly).

2. **Reboot:** Reboot the phone, re-open Shizuku, then relaunch your helper app. Ensure you can re-bind (the permission is sticky, so you may not need to re-grant). Repeat a call test.

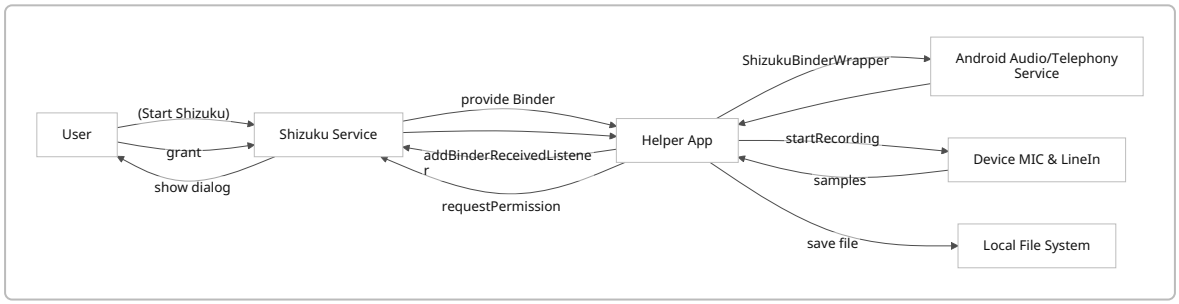
3. **ADB Dumpsys:** After a test call, run:

   ```
   adb shell dumpsys media.audio_flinger
   ```

This shows active recording tracks. You should see an `AudioRecord` client from your app with sampling rate and channels.

4. **Failure modes:** If only one side is recorded, try disabling Wi-Fi calling or switching audio route (e.g. speaker). If Shizuku logs show an error, inspect it. Adjust code as needed.

10. Sample Mermaid Flowchart of Binding Flow



This diagram shows: user starts Shizuku, the helper app registers a listener, Shizuku provides its binder, app requests permission (user grants it), app wraps a system service binder (e.g. AudioService), calls hidden methods to start recording, and audio data flows from hardware to app storage.

11. Sample Logcat Commands

- **Check Shizuku service:**
  `adb shell service list | grep shizuku`
  `adb shell dumpsys activity services moe.shizuku.privileged.api`

- **Monitor app binding/recording:**
  `adb logcat | grep -E "Shizuku|AudioRecord|MediaRecorder"`

- **Filter Shizuku messages:**
  `adb logcat -s Shizuku`

- **Check audio flinger:**
  `adb shell dumpsys media.audio_flinger`

- **General recording logs:**
  `adb logcat -v time | grep Recording`

Use these commands during testing to verify that Shizuku is connected and audio recording is happening as expected.

References

- Shizuku official API documentation (RikkaApps/Shizuku-API)【1†L340-L349】【1†L315-L318】
- HackTricks “Shizuku Privileged API” guide【29†L54-L64】【29†L147-L156】
- APH (ACR Phone Helper) official site (NLL Apps)【3†L30-L39】【3†L59-L67】

- ACR Phone Helper GitHub (NLLAPPS/ACRPhoneHelper) – background info【4†L268-L275】【4†L142-L149】
- ShizuCallRecorder project (Kitsumed) – example of Shizuku-based call recording【44†L310-L319】
- Cally (LyoSU) project README – architecture of Shizuku-based recording【24†L708-L715】【24†L668-L677】
